

gPPM: A Generalized Matrix Operation and Parallel Algorithm to Accelerate the Encoding/Decoding Process of Erasure Codes

SHIYI LI, Harbin Institute of Technology, Shenzhen, Wuhan National Laboratory for Optoelectronics, and Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies, China

QIANG CAO, Huazhong University of Science and Technology and Wuhan National Laboratory for Optoelectronics, China

SHENGGANG WAN, The School of Computer Science and Technology; Huazhong University of Science and Technology, China

WEN XIA, Harbin Institute of Technology, Shenzhen, Department of New Networks, Peng Cheng Laboratory, Shenzhen, and Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies, China

CHANGSHENG XIE, Huazhong University of Science and Technology and Wuhan National Laboratory for Optoelectronics, China

Erasure codes are widely deployed in modern storage systems, leading to frequent usage of their encoding/decoding operations. The encoding/decoding process for erasure codes is generally carried out using the parity-check matrix approach. However, this approach is serial and computationally expensive, mainly due to dealing with matrix operations, which results in low encoding/decoding performance. These drawbacks are particularly evident for newer erasure codes, including SD and LRC codes. To address these limitations, this

This research was partly supported by the Major Key Project of PCL (Grant No. PCL2022A03); the National Natural Science Foundation of China under Grant no. 62172175 and 61972441; Shenzhen Science and Technology Innovation Program under Grant no. RCYX20210609104510007 and no. JCYJ20200109113427092; the Open Project Program of Wuhan National Laboratory for Optoelectronics (No. 2022WNLOKF010); the Young Innovative Talents Project of General Colleges and Universities in Guangdong Province (No. 2022KQNCX159); Natural Science Foundation of Shandong Province (No. ZR2020LZH014); Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies (2022B1212010005). The preliminary manuscript was published in IEEE ICPP'15 [31]. In the journal version, a generalized PPM algorithm is developed to address the limitation of PPM, which is only applicable to asymmetric parity erasure codes. The proposed algorithm not only overcomes this limitation but also enhances the encoding/decoding performance through fine-grained dynamic matrix calculation sequence selection. The generalized PPM algorithm can achieve up to 102.41% improvement over PPM and can also improve the encoding/decoding performance of symmetric parity erasure codes like RS by up to 32.25%. Moreover, we get additional measurement results from our analysis and experiments.

Authors' addresses: S. Li, Harbin Institute of Technology, Shenzhen, Wuhan National Laboratory for Optoelectronics, and Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies, Shenzhen 518055, China; e-mail: lishiyi@hit.edu.cn; Q. Cao and C. Xie, Huazhong University of Science and Technology and Wuhan National Laboratory for Optoelectronics, Wuhan 430074, China; e-mails: {caoqiang, cs_xie}@hust.edu.cn; S. Wan, The School of Computer Science and Technology and Huazhong University of Science and Technology, Wuhan 430074, China; e-mail: sgwan@hust.edu.cn; W. Xia (Corresponding author), Harbin Institute of Technology, Shenzhen, Department of New Networks, Peng Cheng Laboratory, Shenzhen, and Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies, Shenzhen 518055, China; e-mail: xiawen@hit.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

1544-3566/2023/12-ART51 \$15.00

<https://doi.org/10.1145/3625005>

article introduces the *Partitioned and Parallel Matrix (PPM)* algorithm. This algorithm partitions the parity-check matrix, parallelizes encoding/decoding operations, and optimizes calculation sequence to facilitate fast encoding/decoding of these codes. Furthermore, we present a *generalized PPM (gPPM)* algorithm that surpasses PPM in performance by employing fine-grained dynamic matrix calculation sequence selection. Unlike PPM, gPPM is also applicable to erasure codes such as RS code. Experimental results demonstrate that PPM improves the encoding/decoding speed of SD and LRC codes by up to 210.81%. Besides, gPPM achieves up to 102.41% improvement over PPM and 32.25% improvement over RS regarding encoding/decoding speed.

CCS Concepts: • **Computer systems organization** → **Reliability; Redundancy**; • **Information systems** → **Storage recovery strategies**;

Additional Key Words and Phrases: Erasure codes, storage system, fault tolerance, computational cost, performance optimization, parallelism

ACM Reference format:

Shiyi Li, Qiang Cao, Shenggang Wan, Wen Xia, and Changsheng Xie. 2023. gPPM: A Generalized Matrix Operation and Parallel Algorithm to Accelerate the Encoding/Decoding Process of Erasure Codes. *ACM Trans. Arch. Code Optim.* 20, 4, Article 51 (December 2023), 25 pages.

<https://doi.org/10.1145/3625005>

1 INTRODUCTION

Due to their enormous number of disks and nodes, current data center storage systems have widely deployed erasure codes to ensure high data availability in the face of frequent component failures [7, 14, 38, 49]. RAID-6 coding techniques, such as EVENODD [9] and RDP [13], have traditionally been utilized to withstand two simultaneous disk failures. However, for larger storage systems, more robust erasure codes are required to handle multiple concurrent disk failures. Consequently, erasure codes such as RS code [46], Cauchy RS code [11], and STAR code [22] have been developed and deployed. These erasure codes, which we refer to as *Symmetric Parity* erasure codes, produce all parity blocks (called *symmetrical* parity blocks) by calculating the same number of blocks. All of the erasure codes mentioned above are examples of *Symmetric Parity* erasure codes. Such erasure codes are designed for storage systems to protect against the same class of failures, such as device failures.

Storage systems are prone to various types of failures, including sector, disk, machine, and rack failures, which vary in terms of duration, probability, and protection granularity. In order to ensure efficient resilience to such failures, erasure codes have been designed with specialized parity methods. In single-machine environments, storage systems may be vulnerable to device failures and sector failures. One approach to mitigating these failures is to deploy device-level redundancy to protect against device failures and sector-level redundancy to protect against sector failures. By deploying both types of redundancy, these erasure codes [10, 29, 41] can address severe partial disk failures, including latent sector errors [7, 23, 48] and data corruption [8, 24], as well as conventional fault tolerance against complete disk failures. Besides, these erasure codes can avoid the need for device-level redundancy to protect against sector failures, resulting in space savings. Cloud data centers often experience failure events, 90% of which are transient data unavailable (data unavailable with no permanent data loss), resulting in degraded reads [14, 21, 25, 47]. In such situations, special parity is required to handle degraded reads. To address this issue, Microsoft and Facebook propose LRC code ([21, 47] respectively) that use *local parity* to minimize disk I/O, network overhead, and degraded read latency. These erasure codes have been widely adopted by various practical storage systems, including **Windows Azure Storage (WAS)** [12], Amazon's Elastic Compute Cloud [2], and Facebook's cluster [47]. The latest research in this area, referred to as

EC-wide [20], aims to solve the repair, encoding, and update problem in wide stripes using both parity and topology locality. Unlike *Symmetric Parity* erasure codes, not all parity blocks in these erasure codes are *symmetrical*. We denote these erasure codes as *Asymmetric Parity* erasure codes in this article.

Erasures codes have become increasingly popular, and research attention has shifted to *Asymmetric Parity* erasure codes due to their superior performance in various failure scenarios. However, existing *Asymmetric Parity* erasure codes, such as SD code, PMDS code, and LRC code, still rely on the traditional parity-check matrix method [34, 37] for encoding/decoding, which involves multiplying the large parity-check matrix by all surviving data/parity blocks. This approach is strictly serial and coarse-grained, resulting in slow and inefficient encoding/decoding. Our analysis in Section 2 reveals that some faulty blocks in *Asymmetric Parity* erasure codes are *independent faulty blocks* (denoted in Section 2.3) and thus can be directly recovered in parallel. This discovery creates an opportunity to effectively partition the large parity-check matrix into multiple parts and recover the lost data in parallel, resulting in a new fine-grained and parallel process. Moreover, this process offers multiple calculation sequences with different computational cost, allowing for the identification and application of an optimal calculation sequence to reduce the computational cost. Notably, the calculation sequence with the lowest computational cost may vary depending on the parameters of the erasure code.

To this end, we propose an optimization algorithm to improve the encoding/decoding performance of asymmetric parity erasure codes in this article. The algorithm is called the **Partitioned and Parallel Matrix (PPM)** algorithm, automatically partitions the whole parity-check matrix into multiple sub-matrices, determines an optimal calculation sequence at a minimum computational cost, launches threads for processing *independent sub-matrices* (denoted in Section 3.1), and finally merges all recovered faulty data for processing the *remaining sub-matrix* (denoted in Section 3.1). Unlike the traditional encoding/decoding algorithm, PPM exploits the independence of faulty data and benefits from parallelism, and optimizes the calculation sequence to reduce computational cost. However, PPM can only be applied in asymmetric parity erasure codes. Therefore, we also derive a **generalized PPM (gPPM)** algorithm that leverages the changes in multiply and addition operations in encoding and decoding processes to optimize the performance of symmetric parity erasure codes such as RS code. This is achieved through the use of fine-grained dynamic matrix calculation sequence selection. Specifically, the algorithm takes into account the specific characteristic of the erasure code and dynamically determines the most efficient matrix calculation sequence. It is a general method that can be used on a single matrix and can also be used on each sub-matrix of PPM. Therefore, gPPM can achieve performance gains on both PPM and RS code.

- We propose PPM, an optimization algorithm designed to enhance the encoding/decoding performance of asymmetric parity erasure codes. The PPM algorithm employs three key strategies: partitioning the parity-check matrix, parallelizing the encoding/decoding process, and optimizing the calculation sequence. This enables the PPM algorithm to reduce computational cost and take advantage of computational parallelism in the encoding/decoding process.
- We further derive a gPPM algorithm from PPM to accelerate the encoding/decoding process of both symmetric and asymmetric parity erasure codes. The gPPM algorithm can further reduce the computational cost by the fine-grained dynamic selection of matrix calculation sequence according to the specific characteristic of the erasure code.
- We evaluate the performance of the PPM algorithm and the gPPM algorithm, focusing on the efficiency of encoding/decoding speed by integrating them into the encoding/decoding process of SD, RS, and LRC codes. Our experimental results indicate that PPM algorithm leads to a remarkable improvement of up to 210.81% in the encoding/decoding speed of

Table 1. Symbols and Terminologies

Symbols	Description
n	the number of strips/chunks/disks in a stripe
m	the number of parity strips/chunks/disks in a stripe
k	the number of data strips/chunks/disks in a stripe
r	the number of sectors in a strip
s	the number of additional coding sectors in SD code
z	the number of rows with additional faulty sectors
H	the parity-check matrix
R_H	the number of rows of H
C_H	the number of columns of H
$u(M)$	the number of nonzero elements in matrix M
$v(M)$	the number of elements greater than 1 in matrix M
C_i	the number of <i>mult_XORs()</i> by using the i th calculation sequence
C	the normalized operands used to evaluate the computational cost of encoding/decoding process
c	the coefficient of the multiply operation relative to the addition operation
T	the number of threads to conduct the encoding/decoding process
<i>independent faulty block</i>	The faulty block which does not need to be recovered with all other faulty blocks together
<i>dependent faulty block</i>	The faulty block which needs to be recovered with all other faulty blocks together
<i>independent sub-matrix</i>	The new matrix is extracted from H to recover the independent faulty blocks
<i>remaining sub-matrix</i>	The new matrix is extracted from H to recover the dependent faulty blocks

SD and LRC codes. In addition, the gPPM algorithm has further enhanced the performance, leading to an improvement of up to 102.41% compared to PPM algorithm. Additionally, gPPM algorithm improves up to 32.25% in the encoding/decoding speed of RS code.

The rest of this article is organized as follows. Section 2 presents the necessary background and gives the motivation for our article. The design of the PPM algorithm is detailed in Section 3. Section 4 discusses the design of the gPPM algorithm. Then, we conduct experiments and evaluate the performance improvement of PPM and gPPM in Section 5. Section 6 discusses related work. Finally, we conclude this article in Section 7.

2 BACKGROUND AND MOTIVATION

In this section, we provide some background and key observations that motivate our work and facilitate our presentation of PPM in subsequent sections. To facilitate our discussion, we summarize the symbols used in this article in Table 1.

2.1 Symmetric VS. Asymmetric Parity Erasure Codes

Erasure codes are commonly deployed in storage systems to protect data against component failures. The key idea of erasure codes is to generate m parity strips from k data strips. The collection

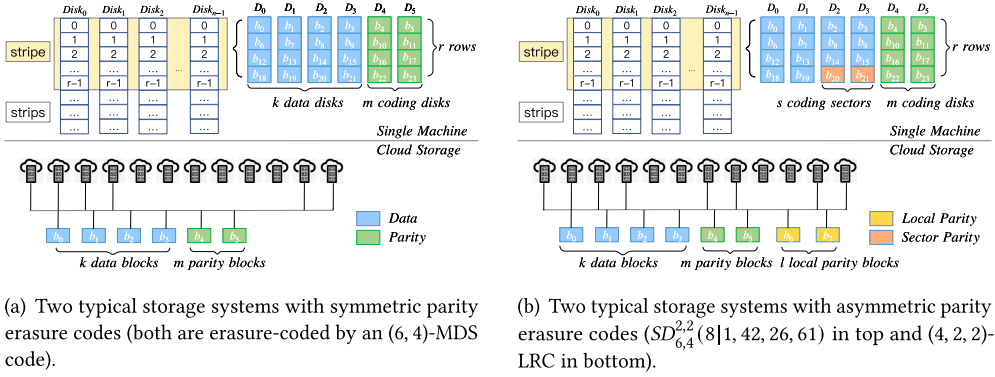


Fig. 1. Storage system with two types of erasure coding. The parameterization of SD code $SD_{6,4}^{2,2}(8|1, 42, 26, 61)$ are denoted in Section 2.3.

of $n = k + m$ strips that encode together is referred to as a stripe. **Maximum Distance Separable (MDS)** codes form a class of erasure codes that possess the property of reconstructing the original data even if m strips fail, known as an (n, k) -MDS code. The entire system can be viewed as a collection of stripes, and each stripe is encoded independently. Thus, we concentrate on a stripe of a storage system in this article. In a stripe, each strip comprises several basic blocks. We refer to the basic blocks as sectors, which may constitute multiple sectors. Since various erasure codes have been presented, they can be classified into different categories in terms of storage cost, parity layout, and so on. We categorize erasure codes from a different perspective in this article. In an erasure code, if each parity block is calculated by k data blocks, it means that all parity blocks are symmetric in terms of the calculation. We denote these erasure codes whose parity blocks are symmetric as *Symmetric Parity* erasure codes. Symmetric parity erasure codes nowadays are inefficient. Transient data unavailable occupy 90% of data center failure events [14, 21, 25, 47]. The data-read will be *degraded* when upper users request these unavailable data. In this case, the requirement for unavailable data triggers a repair process, and once the unavailable data are recovered, the results are returned to the upper users. This procedure is referred to as a *degraded read*. Symmetric parity erasure codes, like Reed-Solomon codes, incur high repair cost in terms of disk I/O and network traffic in degraded read scenarios. Furthermore, large-scale data centers may encounter frequent concurrent complete disk failures and latent sector errors. Plank et al. [41] note that simultaneous disk failures and any additional sector errors are how today's storage systems fail. For symmetric parity erasure codes, it is overkill to dedicate entire disks to tolerate the failure of a few sectors under the combinations of disk and sector failures.

To overcome the shortcomings of symmetric parity erasure codes, newer erasure codes such as SD code, and LRC code are proposed. Unlike symmetric parity erasure codes, these newer codes are designed to have specialized parity for dedicated failures, leading to at least two types of parity blocks. Thus, not all parity blocks are calculated by k data blocks. As shown in Figure 1, each parity block of the $(6, 4)$ -MDS code is calculated by 4 data blocks. However, $(4, 2, 2)$ -LRC code deployed in cloud storage systems dedicates the new parity blocks (called *local parity*) to protect data against degraded read. Hence, parity blocks are categorized into *global parity* and *local parity*. Global parity is similar to traditional parity; therefore, each global parity block is calculated by 4 data blocks. In contrast, each local parity block is calculated by 2 data blocks. In single-machine storage systems, which are erasure-coded with $SD_{6,4}^{2,2}(8|1, 42, 26, 61)$, two additional blocks are devoted to coding, leading to *disk parity* and *sector parity*. Disk parity is the traditional parity calculated by 4 data blocks, while each sector parity block is calculated by 22 blocks. In summary, these newer erasure

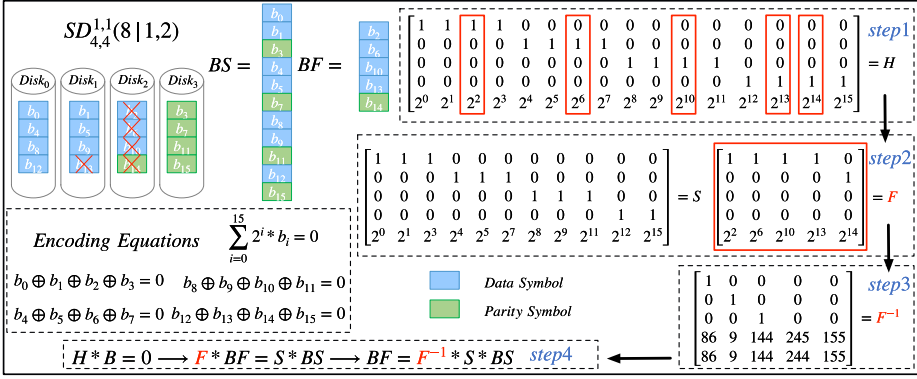


Fig. 2. The encoding/decoding process of traditional encoding/decoding algorithm.

codes design specialized parity for dedicated failures, resulting in asymmetric parity in terms of the calculation. Consequently, we denote these codes as *Asymmetric Parity* erasure codes in this article.

2.2 Traditional Encoding/Decoding Process of Asymmetric Parity Erasure Codes

Despite the existence of asymmetric parity erasure codes, the encoding and decoding performance of these codes still need to be improved. Among the existing asymmetric parity erasure codes, SD code, PMDS code, and LRC code all use a parity-check matrix method [34, 37] to direct the encoding/decoding process. In this article, we take SD code as an example to illustrate the process.¹ Specifically, we parameterize the SD code as $SD_{n,r}^{m,s}(w|a_0, \dots, a_{m+s-1})$, where n is the total number of disks, m is the number of coding disks, s is the number of coding sectors, r is the number of rows per stripe, w is the word size (representing operations over finite fields $\text{GF}(2^w)$), and $a_0, \dots, a_{m+s-1}$ are all coding coefficients. The traditional decoding process typically involves the following four steps:

Step 1: The parity-check matrix H is derived directly from the definition. Assume H is an R_H -row- C_H -column matrix. For SD code, $R_H = m * r + s$, $C_H = n * r$. Each column of H corresponds to a dedicated sector in the stripe. The column $i * n + j$ of H corresponds to the sector b_{i*n+j} in row i and column j , $0 \leq i < r$, $0 \leq j < n$.

Step 2: Two matrices, F and S , are derived from H . The columns corresponding to the faulty blocks are extracted from H to create the matrix F . The remaining columns are extracted from H to construct matrix S .

Step 3: The matrix F is inverted to create the matrix F^{-1} .

Step 4: BF is recovered by calculating $F^{-1} * S * BS$. We denote B as the column vector including all blocks, BF as the column vector including all faulty blocks, and BS as the column vector including all surviving blocks. If there is no faulty block in the stripe, the product of H and B is equivalent to zero ($H * B = 0$). Thus, BF is equal to $F^{-1} * S * BS$. The process from Step 2 to Step 4 is denoted as *matrix decoding*.

In Figure 2, we present an example of the decoding process for $SD_{4,4}^{1,1}(8|1,2)$. The disk array comprises four disks, and the stripe is defined by five equations displayed in the middle of the figure. There are 16 sectors in the example stripe, including five faulty sectors (b_2, b_6, b_{10}, b_{13} , and b_{14}) and eleven surviving sectors. Our analysis of the traditional decoding process of asymmetric parity

¹Since the encoding process of an erasure code is a special case of the decoding process, we focus solely on analyzing the decoding process in this article.

erasure codes reveals that it treats all faulty blocks as a single unit and recovers them together, executing the decoding process serially. However, not all faulty blocks need to be recovered together, as some can be independently recovered from surviving blocks or only require recovery with specific other faulty blocks. In this article, we denote these faulty blocks, which do not need to be recovered with all other faulty blocks together, as *independent faulty blocks* while the remaining faulty blocks as *dependent faulty blocks*. The independent faulty blocks can be recovered first and processed in parallel. Then, all the recovered independent faulty blocks participate in recovering the remaining faulty blocks. As shown in Figure 2, the five faulty sectors, b_2 , b_6 , b_{10} , b_{13} , and b_{14} , are recovered together in *Step 4*. Actually, the three faulty sectors, b_2 , b_6 , and b_{10} , are independent faulty sectors and can be independently recovered from surviving sectors. The other two faulty sectors, b_{13} and b_{14} , are dependent faulty sectors, which not only depend on each other but also on the other faulty blocks, b_2 , b_6 , and b_{10} . Thus, the three faulty sectors, b_2 , b_6 , and b_{10} , can be recovered first and processed in parallel, then we use them to recover the remaining two faulty sectors, b_{13} and b_{14} .

2.3 Observations

Now, we analyze the computational cost of the traditional decoding algorithm. All the addition and multiply operations shown in the article are arithmetic over $\text{GF}(2^w)$, in which linear arithmetic can be decomposed into the basic operations *mult_XORs()* [42]. Similar to Reference [29], we define *mult_XORs*(d_0, d_1, a) as an operation that first multiplies a region d_0 of bytes by a w -bit constant a in Galois Field $\text{GF}(2^w)$ and then applies XOR-summing to the product and the target region d_1 of the same size. For example, $R = a_0 * d_0 + a_1 * d_1 + a_2 * d_2$ can be decomposed into three *mult_XORs()* (assuming R is initialized as zero): *mult_XORs*(d_0, R, a_0), *mult_XORs*(d_1, R, a_1), and *mult_XORs*(d_2, R, a_2). Clearly, fewer *mult_XORs()* bring lower computational cost. To evaluate the computational cost of the encoding/decoding process, we count its number of *mult_XORs()* (per stripe), denoted as C .

As shown in Figure 2, the computational cost of the encoding/decoding process equals the cost of calculating the product of F^{-1} , S , and BS . There are two sequences to calculate $F^{-1} * S * BS$, one where we first multiply S by BS and then multiply F^{-1} by the resulting product of S and BS , denoted as *Normal sequence*, as used in Reference [40]. The other sequence is where we multiply matrix F^{-1} by matrix S first and then multiply the resulting matrix by BS , denoted as *Matrix_First sequence*. The *matrix_first* sequence corresponds to the generator matrix encoding/decoding method [39, 43]. The computational cost of the normal sequence is denoted as C_1 and is determined by the number of nonzero coefficients in matrix F^{-1} and S . The computational cost of the *matrix_first* sequence is denoted as C_2 and is determined by the number of nonzero coefficients in matrix $F^{-1} * S$.² If we denote $u(M)$ as the number of nonzero coefficients in matrix M , we have:

$$C_1 = u(F^{-1}) + u(S), \quad C_2 = u(F^{-1} * S).$$

In general, C_1 does not equal C_2 . For example, Figure 2 shows $C_1 = 35$. Based on the matrix F^{-1} and S shown in Figure 2, we have:

$$F^{-1} * S = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 205 & 125 & 250 & 76 & 131 & 27 & 180 & 216 & 173 & 143 \\ 205 & 125 & 250 & 76 & 131 & 27 & 180 & 216 & 173 & 142 \end{pmatrix}$$

²The computational cost of matrix multiplication is too low ($w \leq 4$ bytes) compared to that of multiplying matrix to block ($\text{sizeof}(\text{block}) \geq 512$ bytes) and thus can be ignored in the analysis.

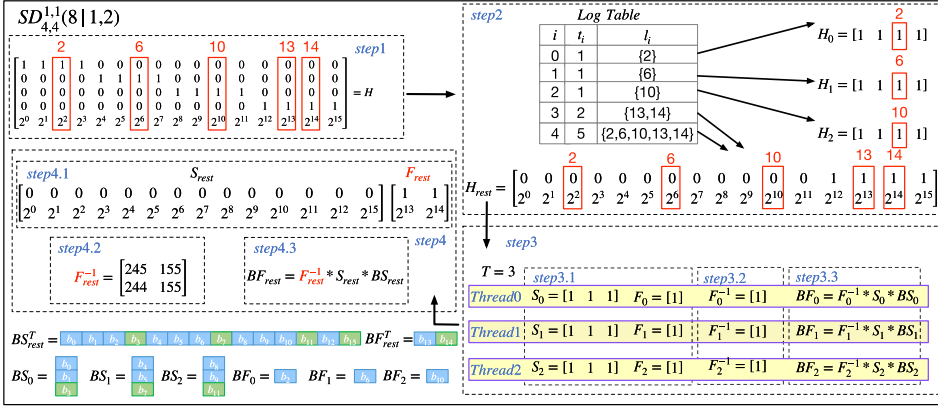


Fig. 3. The encoding/decoding process of the PPM algorithm.

Thus, we can see that $(C_2 = 31) < C_1$. Consequently, different calculation sequences result in different computational cost during the matrix decoding. Furthermore, if we recover the independent faulty blocks first, and then use them to recover the remaining dependent blocks, we can further reduce the computational cost, as shown in Figure 3. The corresponding computational cost is denoted as C_{min} , determined by the number of nonzero coefficients in the sub-matrices $F_0^{-1} * S_0$, $F_1^{-1} * S_1$, $F_2^{-1} * S_2$, F_{rest}^{-1} , and S_{rest} . That is:

$$C_{min} = \sum_{i=0}^2 u(F_i^{-1} * S_i) + u(F_{rest}^{-1}) + u(S_{rest})$$

It is evident that $C_{min} = 29$ is less than $C_2 = 31$, which means that the computational cost can be further reduced by recovering the independent faulty blocks first. In later analysis, we will see that in many cases, the reduction in computational cost is much greater than in this example. In addition, we will describe in the next section how to obtain the optimal computational cost as the parameters change.

We have analyzed the traditional encoding/decoding process of asymmetric parity erasure codes and made two observations: **First**, some faulty blocks are independent faulty blocks and thus can be directly recovered in parallel. **Second**, one can reduce the computational cost by choosing a specific calculation sequence and recovering the independent faulty blocks first. Based on these two observations, we propose our improved algorithms, which will be described in later sections.

3 DESIGN OF PPM

In this section, we will provide a detailed description of the PPM algorithm, which exploits parallelism and optimizes the calculation sequence to reduce the computational cost.

3.1 Independence Exploit and Partition

As mentioned in Section 2.3, some faulty blocks are independent faulty blocks. We will describe how to exploit independence in this subsection. Once these independent faulty blocks are found, we can conduct the partition by extracting the corresponding rows from H .

In order to exploit the independence, we introduce a data structure called *Log Table*. Each row of the log table has the form of (i, t_i, l_i) , where i denotes the row number in H , t_i denotes the number of nonzero elements corresponding to the faulty blocks, and l_i denotes the set consisting of the column numbers of the t_i elements. That is, l_i is denoted as $(j_1, \dots, j_{t_i})$. Thus, the log table has R_H rows. We use $H(i, j)$ to represent the element in row i and column j of H . As shown in Figure 3,

in row 0, only $H(0, 2)$ is not equal to zero and corresponds to faulty sector b_2 . Thus, $t_0 = 1$ and $l_0 = (j_1) = (2)$. The first row of the log table is $(0, 1, (2))$. It is worth mentioning that the reliability of the log table is not a concern as we can regenerate it from matrix H , even if it is lost.

Now, we exploit the independence based on the log table. For each row i of the log table, if $t_i = 1$, the faulty block b_{j_i} is an independent faulty block, $0 \leq i < R_H$. Therefore, the row i is extracted from H to create a new matrix to recover the independent faulty block b_{j_i} independently. The new matrix is denoted as an *independent sub-matrix* in this article. However, if $t_i = f > 1$, we will find whether there are $f - 1$ rows $r_0, \dots, r_{f-2}$ matching $t_{r_0} = \dots = t_{r_{f-2}} = t_i$ and $l_{r_0} = \dots = l_{r_{f-2}} = l_i$, $0 \leq r_0, \dots, r_{f-2} < R_H$, $1 < f < R_H$. If so, the f faulty blocks $(b_{j_1}, \dots, b_{j_f})$ are independent faulty blocks. Besides, each faulty block among this faulty block group depends only on the other $f - 1$ faulty blocks in the same group. The f rows are extracted from H to create an independent sub-matrix to recover the f faulty blocks independently. After traversing all rows, the p independent sub-matrices can be generated. All remaining rows of H are used to create the matrix H_{rest} to recover the remaining faulty blocks. The matrix H_{rest} is denoted as the *remaining sub-matrix* in this article. Thus, H is partitioned into $p + 1$ sub-matrices in total, and p of the $p + 1$ sub-matrices (denoted as $H_0, \dots, H_{p-1}$) are independent sub-matrices that can be decoded in parallel.

Figure 3 illustrates the generation of a five-row log table based on the failure scenario and the parity-check matrix H . In this case, $t_0 = t_1 = t_2 = 1$, indicating that faulty sectors b_2 , b_6 , and b_{10} are all independent faulty sectors. Thus, rows 0, 1, and 2 are extracted from H to create three independent sub-matrices H_0 , H_1 , and H_2 . No row matches $t_i = 2$, $l_i = (13, 14)$ except for row 3, while no row matches $t_i = 5$, $l_i = (2, 6, 10, 13, 14)$ except for row 4, $0 \leq i < 5$. Thus, the two rows, 3 and 4, cannot be extracted from H and are used to create the remaining sub-matrix H_{rest} . The parity-check matrix H is partitioned into four sub-matrices in total, and p is equal to 3. Because the partition operation may cause all zero columns in some sub-matrices, all sub-matrices do not include all zero columns.

The abovementioned approach is a general method to exploit the independence in asymmetric parity erasure codes. Actually, the method to exploit the independence in erasure codes such as SD code, PMDS code, and LRC code are much simpler. Let SD code as an example; we only need to count the number of faulty sectors v in each row i of the stripe, $0 \leq i < r$. If $v = m$, the v faulty sectors are independent faulty sectors, and the corresponding rows are extracted from H to create an independent sub-matrix to recover the v faulty sectors. After traversing all rows, all remaining rows of H are used to create the remaining sub-matrix H_{rest} to recover the remaining faulty sectors.

3.2 Calculation Sequences

Now, we analyze the computational cost of PPM. The computational cost of the encoding/decoding process is the computational cost of the $p + 1$ matrix decoding operations to decode the p independent sub-matrices $H_0, \dots, H_{p-1}$, and the remaining sub-matrix H_{rest} . Thus, it is equal to the cost of calculating the $p + 1$ products: $F_0^{-1} * S_0 * BS_0, \dots, F_{p-1}^{-1} * S_{p-1} * BS_{p-1}$, and $F_{rest}^{-1} * S_{rest} * BS_{rest}$. As shown in Figure 3, the computational cost of decoding $SD_{4,4}^{1,1}(8|1, 2)$ equals the cost of calculating the four products: $F_0^{-1} * S_0 * BS_0, F_1^{-1} * S_1 * BS_1, F_2^{-1} * S_2 * BS_2$, and $F_{rest}^{-1} * S_{rest} * BS_{rest}$.

As mentioned in Section 2.3, different calculation sequences result in different computational cost during the matrix decoding. There are two calculation sequences to decode the large parity-check matrix H in the traditional encoding/decoding process. Because PPM has partitioned H into $p + 1$ sub-matrices, there are 2^{p+1} calculation sequences to decode the $p + 1$ sub-matrices. However, because all elements in the p independent sub-matrices $H_0, \dots, H_{p-1}$, corresponding

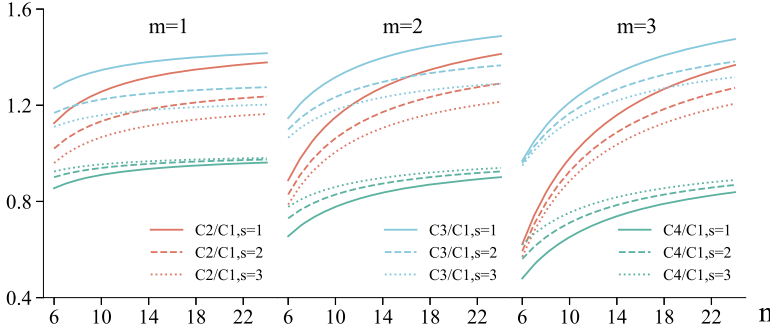


Fig. 4. The computational cost of various calculation sequences, C_i , $1 \leq i \leq 4$, let C_1 as baseline.

to the independent faulty blocks, are not equal to 0, we can derive the following equation, $u(F_i) + u(S_i) > u(F_i * S_i)$. All the p matrix decoding operations on the p independent sub-matrices have lower computational cost by choosing the matrix_first sequence. Consequently, the optional calculation sequences are reduced to 2, which are listed below.

$$C_3 = \sum_{i=0}^{p-1} u(F_i^{-1} * S_i) + u(F_{rest}^{-1} * S_{rest}), \quad C_4 = \sum_{i=0}^{p-1} u(F_i^{-1} * S_i) + u(F_{rest}^{-1}) + u(S_{rest}).$$

Considering an instance of erasure codes, the parity-check matrix H is determined. For an erasure code, the computational cost can be obtained by numerical analysis. For example, if the s additional faulty sectors are located in z rows in the SD code, we can conclude that:³

$$C_1 = n * r * (m + s) + m * (m * r + s) * (z - 1) + m^2 * (r - z),$$

$$C_2 = (n * r - (m * r + s)) * (m * z + s) + m * (n - m) * (r - z),$$

$$C_3 = (n * r - (m + s)) * (m * z + s) + m * (n - m) * (r - z),$$

$$C_4 = n * r * (m + s) + m * (m * z + s) * (z - 1) - m^2 * (r - z),$$

with $4 \leq n \leq 24$, $4 \leq r \leq 24$, $1 \leq m \leq 3$, $1 \leq s \leq 3$, and $1 \leq z \leq s$.

We can see that:

$$C_1 - C_4 = m^2 * (z + 1) * (r - 1) > 0, \quad C_3 - C_2 = m * (r - 1) * (m * z + s) > 0.$$

Thus, the values of C_2 and C_4 are smaller among C_1 , C_2 , C_3 , and C_4 . We only need to choose the smaller one between C_2 and C_4 . However, when choosing C_2 ($C_4 > C_2$), the decoding process just changes the normal sequence to the matrix_first sequence to obtain the optimization. Still, it uses the large parity-check matrix H to conduct the decoding process. Actually, we evaluate the computational cost of SD code under various configurations and failure scenarios and find that the possibility of $C_4 > C_2$ is only around 5%. Besides, the value of n is often equal to 4 or 5 and no more than 9 when $C_4 > C_2$. Thus, we generally choose C_4 except for some special cases when $n \geq 6$, while the reduced computational cost is equal to $C_1 - C_4 = m^2 * (z + 1) * (r - z)$ when compared to the traditional method C_1 . The computational cost is reduced by 20.69% compared to the traditional method C_1 in the example shown in Figure 2.

In Figure 4, we present the computational cost of various calculation sequences for different values of n for SD code when $r = 16$, $z = 1$. As shown in Figure 4, C_4 has the smallest value in most cases. We observe that the values of $\frac{C_2}{C_1}$, $\frac{C_3}{C_1}$, and $\frac{C_4}{C_1}$ become larger with the increased value

³They are derived by the simulation results of Figures 4–7 (print the number of nonzero elements in each matrix and sum them).

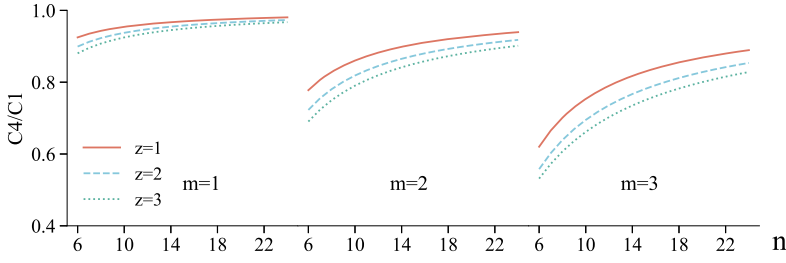


Fig. 5. The value of $\frac{C_4}{C_1}$ for different values of z ($s = 3, r = 16, 1 \leq z \leq s$).

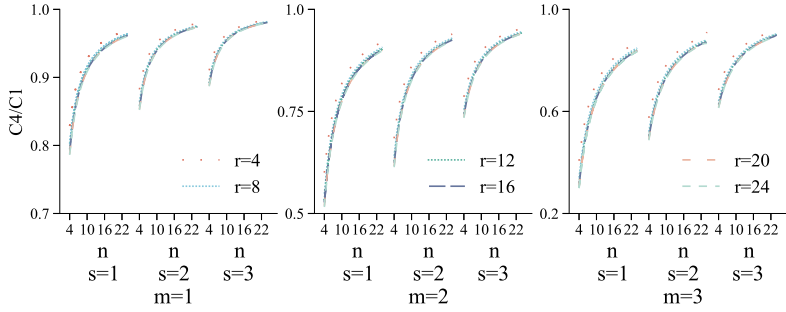


Fig. 6. The value of $\frac{C_4}{C_1}$ for different values of r .

of n and s . They increase more quickly as the increased value of m . The average $\frac{C_4}{C_1}$ is 85.78% (from 47.97% to 98.06%), which is a significant difference. Besides, the value of $\frac{C_4}{C_1}$ becomes larger as the reduced value of z (see Figure 5, where $1 \leq z \leq s$). Similarly, the value of $\frac{C_4}{C_1}$ becomes larger as the reduced value of r (see Figure 6, where $4 \leq r \leq 24$). We observe that the value of $\frac{C_4}{C_1}$ decreases as the value of either z or r increases. Therefore, PPM can reduce the computational cost by selecting the optimal calculation sequence.

3.3 Parallelism

As mentioned in Section 3.1, the partition operation generates p independent sub-matrices, which correspond to the independent faulty blocks. The p independent sub-matrices $H_0, \dots, H_{p-1}$ can be decoded in parallel to recover the corresponding independent faulty blocks. The remaining sub-matrix H_{rest} is decoded after the p matrix decoding operations have finished. As shown in Figure 3, the three independent sub-matrices H_0, H_1 , and H_2 are decoded in parallel to recover the three independent faulty blocks, b_2, b_6 , and b_{10} . After the three matrix decoding operations are completed, the remaining sub-matrix H_{rest} is decoded to recover the remaining two faulty blocks, b_{13} and b_{14} . Thus, the parallelism of the PPM algorithm depends on the value of p .

- **case 1:** $p = 0$, which means that no independent sub-matrix is created, $H_{rest} = H$. In this case, parallelism is not triggered.
- **case 2:** $p = 1$, which means that only one independent sub-matrix is created, and parallelism cannot be triggered.
- **case 3:** $1 < p < R_H$, which means that the degree of parallelism equals p . There are two subcases depending on whether H_{rest} is null.
 - **case 3.1:** if $H_{rest} = \text{NULL}$, there are no dependent faulty blocks.
 - **case 3.2:** if $H_{rest} \neq \text{NULL}$, the common case is processed by PPM.

- **case 4:** $p = R_H$, which means that every faulty sector is independent, and there are no dependent faulty sectors. In this case, $H_{rest} = NULL$, and maximum parallelism is achieved.

We focus on *case 3.2* in this article. The time cost of decoding sub-matrix H_i by `matrix_first` sequence decoding is denoted as c_i , $0 \leq i < p$. Matrix H_{max} represents the sub-matrix with the maximum time cost in decoding among the p sub-matrices $H_0, \dots, H_{p-1}$, which correspond to the independent faulty blocks. Thus, $c_{max} = \max\{c_0, \dots, c_{p-1}\}$, $0 \leq c_{max} < p$. Ideally, the saved time cost is equal to $\sum_{i=0}^{p-1} c_i - c_{max}$. Although some additional time is spent creating multiple threads and conducting matrix multiplication, this latency is relatively low when the sector size is large. We take into consideration the overhead introduced by multi-threading in our experimental results presented in Section 5, and we also limit the number of threads T ($T \leq p$) to avoid thread overloading.

3.4 Partitioned and Parallel Matrix Algorithm

In summary, the PPM algorithm exploits the independence of faulty blocks by partitioning the large parity-check matrix into $p + 1$ sub-matrices, and using T ($T \leq p$) threads to process the p independent sub-matrices in parallel. The recovered faulty blocks are then used to recover the remaining faulty blocks by decoding the remaining sub-matrix H_{rest} , while optimizing the calculation sequence for each matrix decoding operation. Therefore, the decoding process of the PPM algorithm can be summarized as follows:

Step 1: Derive the parity-check matrix H .

Step 2: Generate a log table based on the failure scenario and H , and partition H into $p + 1$ sub-matrices based on the log table.

Step 3: Arrange T ($T \leq p$) threads to decode the p independent sub-matrices in parallel using the `matrix_first` sequence for each independent sub-matrix H_i , $0 \leq i < p$.

In this step, two new matrices F_i and S_i are derived from each independent sub-matrix H_i (labeled with *Step 3.1*), matrix F_i is inverted to create the matrix F_i^{-1} (labeled with *Step 3.2*), and the product of F_i^{-1} , S_i , and BS_i (labeled with *Step 3.3*) is calculated using the `matrix_first` sequence, $0 \leq i < p$.

Step 4: Use the recovered independent faulty sectors to recover the remaining faulty sectors by decoding the remaining sub-matrix H_{rest} using the normal sequence. Only the columns corresponding to the remaining faulty sectors from H_{rest} are extracted to create the matrix F_{rest} , and the remaining columns of H_{rest} are used to create S_{rest} . Once all of the faulty sectors are recovered, the decoding process is complete.

In this section, we continue to use the example of $SD_{4,4}^{1,1}(8|1, 2)$ to illustrate the decoding process of PPM algorithm, as shown in Figure 3. The failure scenario remains unchanged. As mentioned earlier in Section 3.1, SD code has a much simpler method to exploit the independence. Algorithm 1 describes the decoding process of PPM for SD code.

In summary, the PPM algorithm offers two key advantages. Firstly, it exploits the parallelism of multiple concurrent threads to enable decoding operations to be performed simultaneously, thereby improving overall performance. Secondly, it reduces the computational cost by partitioning the matrix and optimizing the calculation sequence, thereby achieving better efficiency in decoding.

4 THE EXTENSION OF PPM

We have proposed the PPM algorithm to improve the encoding/decoding performance of asymmetric parity erasure codes such as SD code. Further optimization of the algorithm and its application to improve the performance of symmetric parity erasure codes, such as RS code, will be discussed.

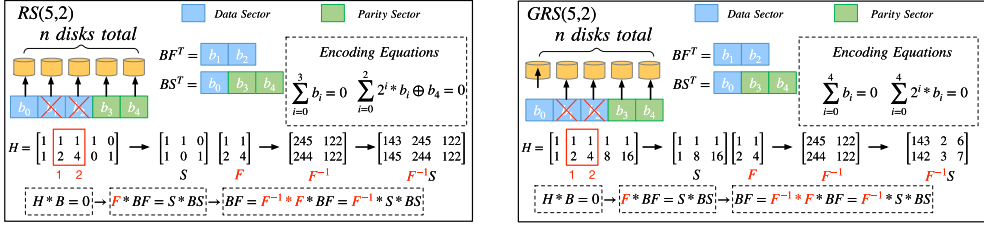


Fig. 7. The matrix_first and normal sequences encoding/decoding process of RS(5, 2) and GRS(5, 2).

ALGORITHM 1: PPM decoding algorithm for SD code.

Step 1: Given an instance of $SD_{n,r}^{m,s}(w|a_0, \dots, a_{m+s-1})$, we derive the parity-check matrix H directly from the definition, H is $m * r + s$ -row- $r * n$ -column array.

Step 2: Partition H into $p + 1$ sub-matrices and p of the $p + 1$ sub-matrices are independent sub-matrices which triggers T threads to decode them in parallel.

Let $p \leftarrow 0$;

Let $string \leftarrow NULL$;

Create the remaining sub-matrix H_{rest} ;

for $i = 0$ **to** $r - 1$ **do**

Let $c \leftarrow 0$;

for $j = 0$ **to** $n - 1$ **do**

if $NULL == (string = read(b_{i*n+j}))$ **then**
 $c++$;

if $c > m$ **then**

Use the $m * i, m * i + 1, \dots, m * i + m - 1$ th row of H to create independent sub-matrix H_p .
 Arrange thread $(p \bmod T)$ to decode the independent sub-matrix H_p by using matrix_first sequence decoding.

$p++$;

else

Use the $m * i, m * i + 1, \dots, m * i + m - 1$ th row of H to fill up the remaining sub-matrix H_{rest} .

Step 3: Once the T threads used to decode the p independent sub-matrices $H_0, \dots, H_{p-1}$ are finished, we decode the remaining sub-matrix H_{rest} by using normal sequence decoding.

Matrix Decoding:

Substep 1: Create two new matrices S_p and F_p . Use these columns of H_p which correspond to the faulty sectors to create F_p and the remaining columns of H_p to create S_p .

Substep 2: Invert matrix F_p to create the matrix F_p^{-1} .

Substep 3: Calculate the product of F^{-1} , S and BS . The result is the value of the recovered faulty sectors.

4.1 RS and GRS Examples

In this section, we evaluate the performance of $RS(n, m)$ code for two examples, $RS(5, 2)$ and $RS(15, 2)$, using both normal and matrix_first sequence encoding/decoding processes. The encoding/decoding process for $RS(5, 2)$ and $RS(15, 2)$ is shown in Figures 7 and 8. In order to calculate $F^{-1} * S * BS$, the normal sequence is that we multiply S by BS first and then multiply F^{-1} by the product of S and BS . Matrix_first sequence is that we multiply matrix F^{-1} by matrix S first and then multiply the result by BS . Accordingly, the encoding/decoding performance of $RS(5, 2)$ is 2389.67MB/s (normal sequence) and 2792.81MB/s (matrix_first sequence), whereas for $RS(15, 2)$, the performance is 3387.03MB/s (normal sequence) and 2836.9MB/s (matrix_first sequence). It is

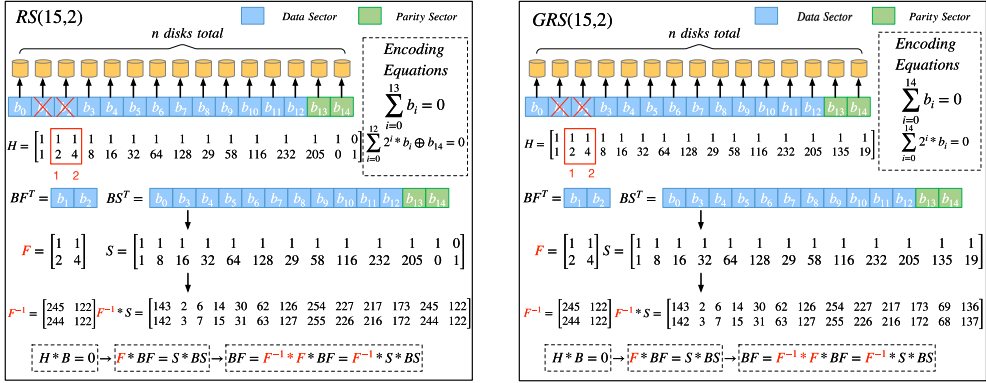


Fig. 8. The matrix_first and normal sequences encoding/decoding process of RS(15, 2) and GRS(15, 2).

observed that the matrix_first sequence is better for RS(5, 2) but worse for RS(15, 2). The experimental setup is described in the following section.

The observed phenomenon where the matrix_first sequence performs better for RS(5, 2) and worse for RS(15, 2) can be explained by the fact that although the matrix_first sequence has fewer nonzero elements in the matrices (i.e., $u(F^{-1} * S) < u(F^{-1}) + u(S)$), it has more non-one elements (i.e., $v(F^{-1} * S) > v(F^{-1}) + v(S)$). We denote the number of elements greater than 1 in the matrix F as $v(F)$. The presence of non-one elements in the matrices leads to more multiply operations, which are slower than addition operations. Specifically, in the case of RS(5, 2), as shown in Figure 7:

$$(u(F^{-1} * S) = 6) < (u(F^{-1}) + u(S) = 8), \quad (v(F^{-1} * S) = 6) > (v(F^{-1}) + v(S) = 4),$$

while in the case of RS(15, 2), as shown in Figure 8:

$$(u(F^{-1} * S) = 26) < ((u(F^{-1}) + u(S) = 28), \quad (v(F^{-1} * S) = 26) > (v(F^{-1}) + v(S) = 14).$$

The RS code we discussed above is a standard version. A general version of RS code (denoted as GRS) is used as a part of the SD code. The difference between RS and GRS lies in whether the parity blocks are calculated independently and the parameters corresponding to the parity blocks. Figures 7 and 8 also provide examples of GRS(5, 2) and GRS(15, 2). The encoding/decoding performance of GRS(5, 2) is 1890.43MB/s (normal sequence) and 2792.61MB/s (matrix_first sequence). However, the encoding/decoding performance of GRS(15, 2) is 3142.15MB/s (normal sequence) and 2826.76MB/s (matrix_first sequence). The GRS version is similar to the standard RS but with different gap values. In the case of GRS(5, 2), as shown in Figure 7, we can conclude that:

$$(u(F^{-1} * S) = 6) < (u(F^{-1}) + u(S) = 10), \quad (v(F^{-1} * S) = 6) == (v(F^{-1}) + v(S) = 6),$$

while in the case of GRS(15, 2), as shown in Figure 8:

$$(u(F^{-1} * S) = 26) < (u(F^{-1}) + u(S) = 30), \quad (v(F^{-1} * S) = 26) > (v(F^{-1}) + v(S) = 16).$$

And then the computational cost can be denoted as follows:

$$C = u + c * v.$$

Accordingly,

$$C(F^{-1} * S) = u(F^{-1} * S) + c * v(F^{-1} * S), \quad C(F^{-1} + S) = (u(F^{-1}) + u(S)) + c * (v(F^{-1}) + v(S)),$$

where c is the coefficient of the multiply operation relative to the addition operation.

Figure 9 provides a visualization of the number of addition and multiplication operations triggered by different encoding/decoding sequences for various RS and GRS parameters. The results indicate that the matrix_first sequence reduces the number of addition operations, particularly when n is small, for both RS and GRS. However, the gap between the matrix_first sequence and the normal sequence in terms of addition operations is more significant in GRS than in RS. Additionally, in GRS, the matrix_first sequence requires fewer multiplication operations compared

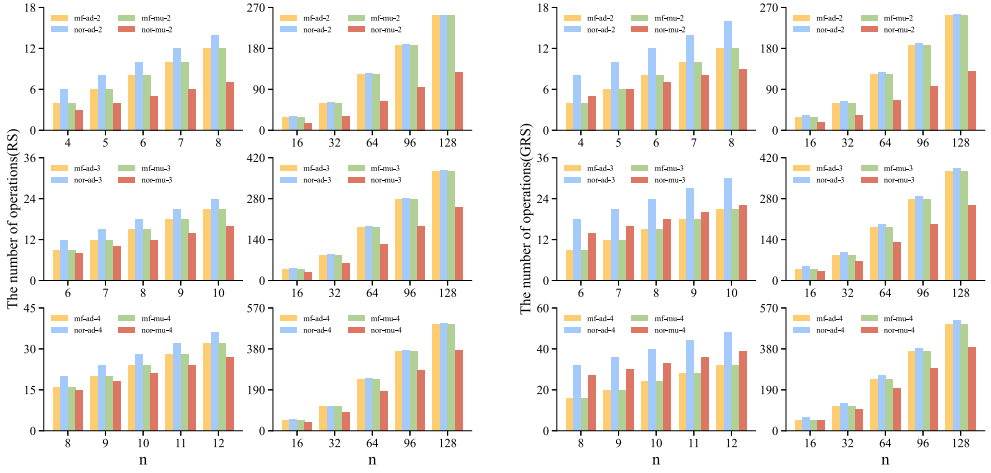


Fig. 9. The number of addition and multiply operations triggered by the encoding/decoding process of RS and GRS. mf: matrix_first sequence. nor: normal sequence. ad: addition operation. mu: multiply operation. $m=2,3,4$. Thus mf-ad-2 represents the addition operation of the matrix_first sequence in RS($k,2$) and GRS($k,2$).

to the normal sequence. Conversely, in RS, the matrix_first sequence requires more multiplication operations than the normal sequence. Therefore, the performance improvement of gPPM is more significant in GRS than in RS, as discussed in Section 5.3. Moreover, for large values of n , the normal sequence requires fewer multiplication operations and comparable addition operations for both RS and GRS.

From the above discussion, it is evident that both RS and GRS codes have an optimal calculation sequence to optimize the computational cost of the encoding/decoding process. The optimal sequence, however, varies as the parameters change and can be derived based on $\text{Min}(C(F^{-1} * S), C(F^{-1} + S))$. Whether it is RS or GRS, the choice of the optimal calculation sequence has the potential to improve the performance of the encoding/decoding process greatly.

4.2 Generalized PPM Algorithm

The example in the previous sub-section demonstrates that the computational cost of the encoding/decoding process in symmetric parity erasure codes such as RS code can be further optimized. To achieve this, we need to compute the $\text{Min}(C(F^{-1} * S), C(F^{-1} + S))$ to choose the optimal sequence and obtain better performance. It is worth noting that when $m = 1$ (such as RAID-5), the matrix_first sequence and normal sequence are always the same. Therefore, our focus is on the $m > 1$ scenario (such as RAID-6 and erasure codes with more than two fault tolerance), which is also the primary scenario in modern erasure-coded systems.

In the case of asymmetric parity erasure codes like SD and LRC codes, the sub-matrices take the form of symmetric parity erasure codes after partitioning. Therefore, we can compute $\text{Min}(C(F^{-1} * S), C(F^{-1} + S))$ for each sub-matrix to select the optimal calculation sequence and improve the encoding/decoding performance. Building on this observation and the PPM algorithm, we derive a gPPM algorithm, which is presented in Algorithm 2. The selection of matrix calculation sequence in PPM is coarse-grained, static and does not distinguish between addition and multiplication. In gPPM, the difference between addition and multiplication is used to derive $\text{Min}(C(F^{-1} * S), C(F^{-1} + S))$ as the basis for the selection of matrix calculation sequence. The key idea of gPPM is the fine-grained dynamic matrix calculation sequence selection. It is a general method that can be used on a single matrix and can also be used on each sub-matrix of PPM. Therefore, gPPM can achieve

ALGORITHM 2: Generalized PPM Algorithm

Step 1: Determine whether it is a symmetric parity erasure code. If yes, go to Step 2.

Otherwise, go to Step 3.

if symmetric parity erasure code then

 | go to Step 2

else

 | go to Step 3.

Step 2: Compute the value of $Min_A = Min(C(F^{-1} * S), C(F^{-1} + S))$.

if $Min_A == C(F^{-1} * S)$ then

 | conduct matrix_first sequence encoding/decoding

else

 | conduct normal sequence encoding/decoding.

Step 3: Compute the value of $Min_B = Min(C(F^{-1} * S), C(F^{-1} + S), C_{ppm})$.

if $Min_B == C(F^{-1} * S)$ then

 | conduct matrix_first sequence encoding/decoding.

else if $Min_B == C(F^{-1} + S)$ then

 | conduct normal sequence encoding/decoding.

else

 | go to step 4.

Step 4: Conduct PPM algorithm to encoding/decoding.

Compute the value of

$$C_{ppm} = \sum_{i=0}^{p-1} Min(C(F_i^{-1} * S_i), C(F_i^{-1} + S_i)) + Min(C(F_{rest}^{-1} * S_{rest}), C(F_{rest}^{-1} + S_{rest})).$$

For each sub-matrix ($F_i (0 \leq i < p)$ and F_{rest}), conduct Step 2.

Because the results of Step 2 are all the same for the p sub-matrices $F_i (0 \leq i < p)$. Thus we only need to perform Step 2 once, and the remaining $p - 1$ sub-matrices use the result of the first sub-matrix p_0 .

performance gains on both PPM and RS code. The gPPM algorithm offers two main improvements over the PPM algorithm:

- The gPPM algorithm can further reduce the computational cost of each sub-matrix in PPM by choosing the calculation sequence based on $Min(C(F^{-1} * S), C(F^{-1} + S))$.
- There are 5% cases that the matrix_first sequence without partition is optimal.

In addition, the benefits of the gPPM algorithm are not limited to asymmetric parity erasure codes; it can also be applied to symmetric parity erasure codes to improve their encoding/decoding performance. Moreover, since the calculation sequence does not affect parallelism, the parallel optimization of the gPPM algorithm is the same as that of the PPM algorithm.

5 PERFORMANCE EVALUATION

In Section 3, we presented the design of the PPM algorithm and analyzed its reduced computational cost from a theoretical perspective. In the previous section, we introduced the gPPM algorithm, which can provide further improvements in encoding/decoding performance compared to PPM and also benefit symmetric parity erasure codes such as RS code. In this section, we will evaluate the performance of these algorithms through real experiments.

5.1 Experiment Setup

We run our performance experiments on three machines equipped with Intel E5 – 2603 (1.80 GHz, 10 MB L3-cache, 4-cores), i7 – 3930K (3.20 GHz, 12 MB L3-cache, 6-cores), and E5 – 2650 (2.00 GHz, 20 MB L3-cache, 8-cores) respectively. All CPUs have a 256KB L2-cache and support SSE4.2. Each decoding operation runs 10 times; the average results are shown in the following figures. All

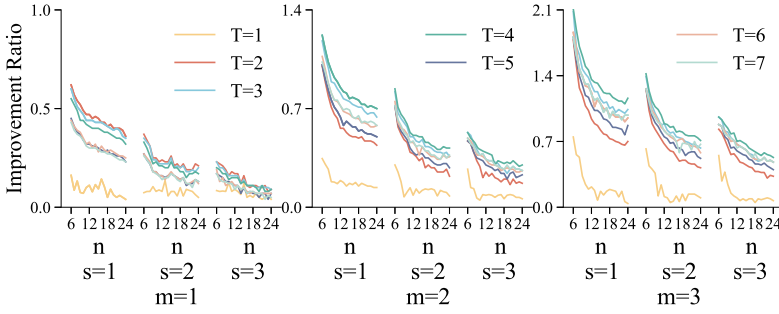


Fig. 10. The performance improvement of PPM for SD code under different values of T (stripe size = 32 MB, $r = 16$, $z = 1$, 4-cores $E5 - 2603$ CPU).

experiments employ Intel's SIMD instruction to accelerate the decoding performance [3, 42]. The optimization of PPM and gPPM are both not affected by the instruction sets. The use of Intel's SIMD instruction is because it is the most mainstream instruction, widely used by academia and industry. We have also tested our algorithm without Intel's SIMD instructions, and although there is a similar performance improvement, the improved speed is still only a few tens of MB/s; it is far from the speed of using Intel's SIMD instruction (GB/s).

5.2 Evaluation of PPM

Implementation: We implement PPM on the SD code, PMDS code, and LRC code to optimize the encoding/decoding performance and evaluate the improvement. And then, the optimized encoding/decoding performance is compared with the RS code. All these codes are based on the Galois Field arithmetic over w -bit symbols. To implement these codes, we make some modifications to the open-source SD encoder and decoder [40]. PPM is written in C and integrated into the encoder and decoder of these codes. Since PMDS code is a subset of SD code, the experimental results of SD code also reflect that of PMDS code.

When a disk fails, the other disks in the array may encounter latent sector errors at the same time [7, 13, 41]. We focus on the scenario of simultaneous disk and sector failures. If the number of faulty disks m' is smaller than m or the number of faulty sectors s' is smaller than s , the decoding process resembles the worst case of decoding $SD_{n,r}^{m',s'}(w|a_0, \dots, a_{m+s-1})$. So, we only test the worst case in this article. We use a random integer generator [45] to simulate the m faulty disks (m random numbers in $(0..n-1)$) and the s additional faulty sectors (the surviving sectors are labeled from 0 to $(n-m) \cdot r - 1$, s random numbers in $(0..(n-m) \cdot r - 1)$). The s additional faulty sectors can reside on z ($1 \leq z \leq s$) rows. We have tested the performance of PPM for different z and found that PPM always achieves nearly the same performance improvement for different values of z . Thus, we only show the results of $z = 1$ in this article.

Parallel: For SD code, the degree of parallelism p is equal to $r - z$. In order to avoid thread overloading, restrain the additional cost of parallelism, and maintain CPU overhead, we set $T \leq \min\{4, \text{corenumbers}\}$. Actually, when the value of T is larger than the *corenumbers*, the improvement will decrease. Figure 10 shows the improvement of PPM under different values of T . The improvement ratio represents $(PPM/SD) - 1$, which is similar in the figures below. Even with only a single thread ($T = 1$), PPM can benefit from the performance optimization of matrix operations, which is consistent with the analysis in Section 3.2. When $T < \text{corenumbers}$, varying T can represent the effect of the different numbers of cores over PPM. When $m > 1$, the improvement of PPM increases as the increased value of T when $T \leq \text{corenumbers}$ and then reverses when $T \geq \text{corenumbers}$. PPM achieves the maximum improvement when $T = \text{corenumbers}$. When

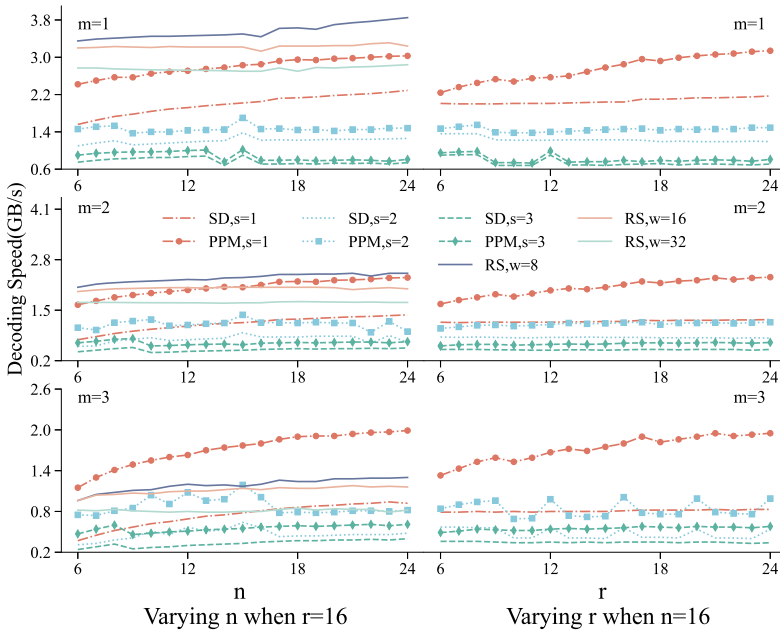


Fig. 11. The decoding speed of PPM for SD code for different values of n and r (stripe size = 32 MB, $T = 4$, $z = 1$, 4-cores E5 – 2603 CPU).

$m = 1$, PPM achieves the maximum improvement at $T = 2$, and the improvement of PPM decreases as the increased value of T . This is because the parallelism part accounts for a small proportion of the entire encoding/decoding process, and the additional cost is relatively high when $m = 1$. That is also the reason why PPM performance improvement does not increase linearly with the number of threads. Besides, the improvement of PPM is more sensitive to the configurations of SD code.

Parameters: Figure 11 shows the performance improvement of PPM for SD code for different values of n and r . The encoding/decoding performance of SD code becomes lower as the value of either m or s increases (close to the speed of disk I/O). Thus, optimizing the encoding/decoding performance becomes more acute. PPM improves the decoding speed by 61.09% on average (from 8.22% to 210.81%). We note that the performance improvement becomes smaller with the increased value of n or s and the decreased value of m or r . It is consistent with the numerical analysis results mentioned in Section 3.2. The jagged lines in all these figures are a result of switching between $GF(2^8)$, $GF(2^{16})$, and $GF(2^{32})$ [41]. Besides, we also test $GF(2^8)$, $GF(2^{16})$, and $GF(2^{32})$ for RS code (Note that all results of RS code shown in the figure are with $m + 1$). The results show that the speed of the PPM with m is competitive to RS code with $m + 1$, especially when the value of either n or m is large.

CPU: In order to verify that the efficiency of PPM is independent of the performance of the CPU, we conduct the experiment on three kinds of CPUs, i7 – 3930K, E5 – 2650, and E5 – 2603, and find that PPM achieves similar improvement on all three CPUs, as shown in Figure 12. The optimization of PPM is only 8.22% when the parameters are $n = 24$, $s = 3$, and $m = 1$. This is because the computational cost optimization is very small. At the same time, in this scenario, the proportion of parallelism in the entire coding process is very small, making the entire parallel optimization not obvious. Besides, the changes in Figure 12 are all caused by changes in computational cost due to parameter changes.

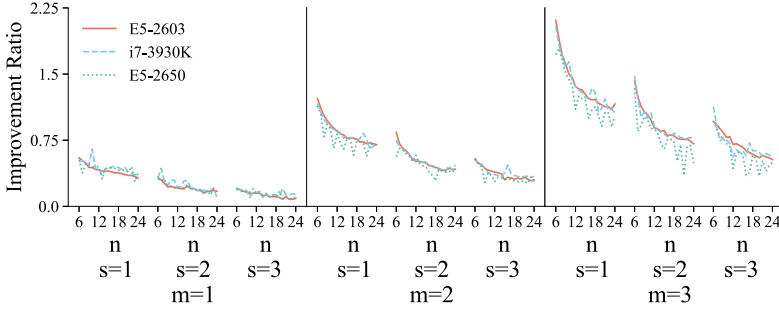


Fig. 12. The performance improvement of PPM for SD code for different CPUs (stripe size = 32 MB, $r = 16$, $T = 4$, $z = 1$).

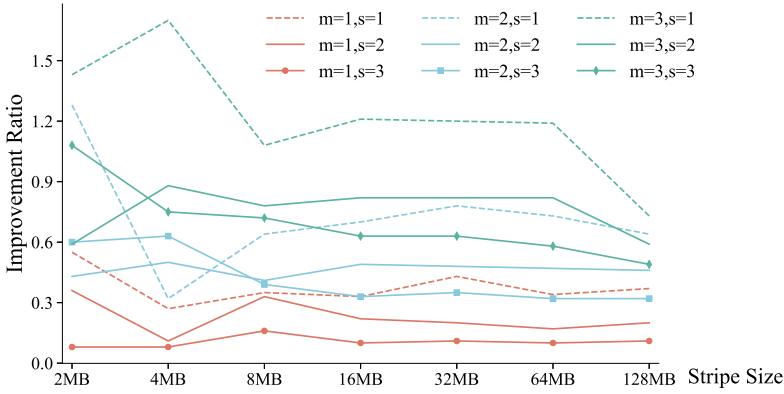


Fig. 13. The performance improvement of PPM for SD code for different stripe sizes ($n = 16$, $r = 16$, $T = 4$, $z = 1$, 4-cores E5 – 2603 CPU).

Stripe Size: In addition, we evaluate the impact of stripe size on the improvement of PPM, as shown in Figure 13. The effect of multi-threading overhead on PPM decreases as the stripe size increases, resulting in steady improvement when the stripe size is larger than 8MB.

LRC: Furthermore, we also execute PPM on LRC code and observe a performance improvement ranging from 16.28% to 36.71% for different storage cost (ranges from 1.1 to 1.7) with fixed stripe size (stripe size = 32MB) and fixed strip size (strip size = 64MB), as shown in Figure 14. However, the improvement achieved by PPM on LRC code is lower than that on SD code due to the latter's higher degree of parallelism, which benefits more from parallelism. Nevertheless, PPM demonstrates the potential to improve performance on asymmetric parity erasure codes.

One of the notable features of PPM is that it utilizes the already available resources without introducing any extra resources to conduct parallelism. Even with just two threads for parallel decoding, PPM can still achieve a significant improvement in performance (46.29% on average, from 8.45% to 178.38%). As such, PPM does not cause thread overloading and does not consume much CPU overhead. The extra power consumption of PPM is also minimal (our test results show that it is no more than two watts). The increase in power consumption of the PPM algorithm (no more than two watts) is caused by parallelism and accounts for a small proportion of the overall machine power (perhaps less than 1%), but the increase in the speed of the entire encoding/decoding can make the overall energy consumption less. In addition, the reduction in computational cost can also reduce energy consumption. Therefore, PPM can reduce computational

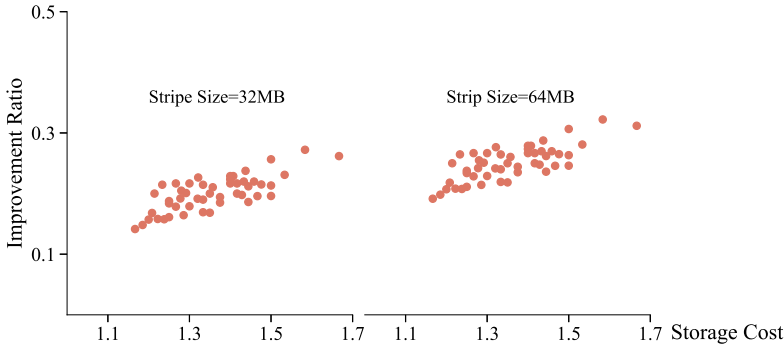


Fig. 14. The performance improvement of PPM for LRC code for different storage cost with fixed stripe size of 32MB and fixed strip size of 64MB (CPU is E5 – 2603).

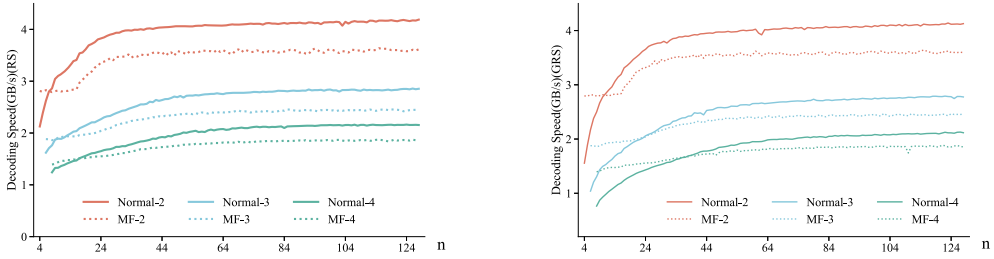


Fig. 15. The encoding/decoding performance of matrix_first sequence and normal sequence in RS and GRS. MF: matrix_first sequence. Normal: normal sequence. $m = 2, 3, 4$. Thus MF-2 represents the decoding performance of the matrix_first sequence in RS($k, 2$) and GRS($k, 2$).

cost and gain benefits in CPU overhead and power/energy efficiency. Additionally, we have demonstrated that PPM consistently achieves performance improvement for different values of n , m , r , s , z , T , stripe sizes, and CPUs for both SD and LRC codes, up to 210.81%.

5.3 Evaluation of GPPM

Implementation: To optimize the encoding/decoding performance of SD and RS codes, we have implemented gPPM and evaluated its improvement. The experimental environment for this evaluation is nearly identical to that used for the evaluation of PPM. gPPM is written in C and integrated into the encoder and decoder of these codes.

RS and GRS: Based on the analysis in Section 4.1, we first evaluate the encoding/decoding performance of matrix_first and normal sequence in RS and GRS, as shown in Figure 15. The results indicate that the matrix_first sequence performs better than the normal sequence when n is small. Conversely, when n is large, the normal sequence is better than the matrix_first sequence. This is because the advantage of the matrix_first sequence in the addition operation is greater than the sacrifice in the multiplication operation when n is small. On the other hand, when n is large, the opposite is true. These findings suggest that for small n , particularly in the range of $4 \leq n \leq 8$, which is commonly used in traditional RAID arrays, the Matrix_first sequence should be used. Conversely, for large n , particularly in wide stripe coding [20, 54, 55], the normal sequence is recommended.

Figure 16 shows the improvement of the gPPM algorithm in symmetric parity erasure codes such as RS and GRS. In RS, the algorithm can provide up to 32.25% optimization, while in GRS,

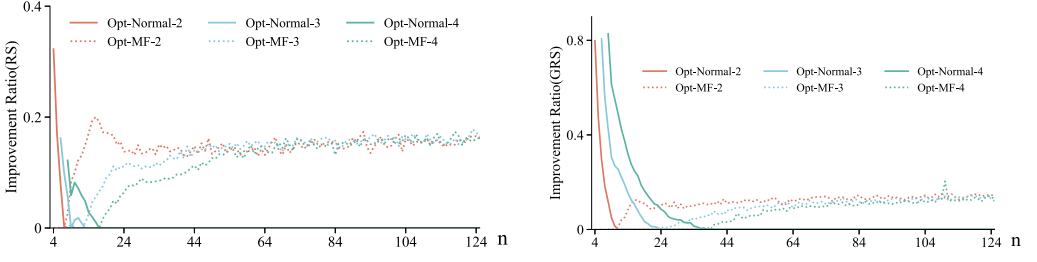


Fig. 16. The performance improvement of the gPPM algorithm in RS and GRS. MF: matrix_first sequence. Normal: normal sequence. $m = 2, 3, 4$. Thus Opt-MF-2 represents the performance improvement of the gPPM algorithm over the matrix_first sequence in RS($k, 2$) and GRS($k, 2$).

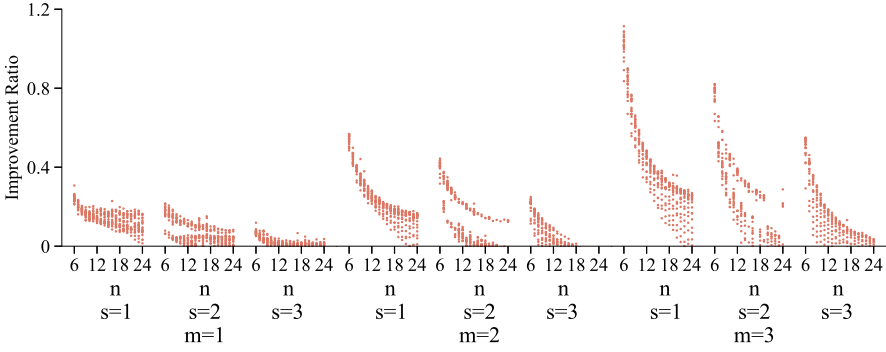


Fig. 17. The performance improvement of the gPPM algorithm over PPM in SD code.

which is similar to RS, but with a larger gap when n is small and a smaller gap when n is large, up to 79.73% optimization can be achieved. Since GRS is used as part of SD, the gPPM algorithm can offer significant improvement over PPM.

gPPM over PPM: In Figure 17, we present the improvement of the gPPM algorithm over PPM in SD code. The x-axis represents the 1323 SD code configurations that have shown performance improvement. It can be observed that by deploying the gPPM algorithms, nearly 1/3 of all SD code configurations have a significant optimization effect, which can be optimized by up to 102.41%. However, PPM remains the optimal algorithm in the remaining 2/3 of all SD code configurations. Notably, all the optimizations achieved by gPPM result from reduced computational cost.

Apart from the PPM sub-case, the computational cost reduction is the primary source of improvement in the gPPM algorithm. The number of addition and multiplication operations depends on the number of non-one and nonzero elements in the intermediate matrices during the encoding and decoding process. Both gPPM and PPM algorithms are effective in reducing computational cost. While gPPM is a general method, PPM applies only to asymmetric parity erasure codes. Combining PPM with gPPM can further improve the performance. This article provides an efficient evaluation of the two approaches, which can significantly enhance the encoding/decoding performance of both asymmetric and symmetric parity erasure codes.

6 RELATED WORK

Speeding up the encoding/decoding process of erasure codes is a critical area of research that has received significant attention [9, 11, 13, 29, 42, 43, 53]. The encoding/decoding procedure is common, and speeding it up has potential advantages on the latency of the system. It is triggered during

reconstruction/full-stripe writes, failures occur [13] (failures happen in bursts [7, 14, 38, 49]), extents are sealed [21], and the users read the data protected by non-systematic code (such as F-MSR [19]). Besides, guaranteeing high-speed data encoding and decoding for erasure coding has become more important with the rapid development and wide adoption of new high performance NVM and NVMe storage devices [1, 4, 5, 26, 27, 44, 56]. The computation time required for encoding and decoding data may not be trivial when compared to the speed of high-performance storage components. It is also the reason behind the researcher's focus on improving the encoding/decoding performance of existing erasure codes [3, 6, 9, 11, 13, 15, 16, 20, 42]. These technologies are presented to improve symmetric parity erasure codes' encoding/decoding speed.

Researchers [9, 11, 13, 29] present newer erasure codes to reduce the computational cost of the encoding/decoding process. EC-wide [20] uses a multi-node encoding scheme to overcome the limitation of single-node encoding to improve the encoding/decoding performance of wide-stripes erasure coding. Actually, EC-wide is a particular extension type of LRC that can also benefit from our gPPM algorithm. Anvin's scheme [6] optimizes RS encoding specifically for RAID-6. Researchers [3, 42] use Intel's SIMD instruction to accelerate multiple operations of Galois Field, which is integrated into our scheme in experiments. Researchers [16] also use general-purpose GPUs to improve encoding/decoding efficiency, but our implementation uses multi-threading instead, which is more general and does not introduce additional cost.

However, only a few studies have been conducted to analyze the matrix encoding/decoding process and improve erasure codes' encoding/decoding performance. PPM [31] is the first general algorithm that optimizes asymmetric parity erasure codes' encoding/decoding performance. We derive gPPM from PPM to further improve the encoding/decoding performance, and it can be applied to RS code. APCM [15] uses a matrix transformation process to avoid matrix inversion computations and reduce the computational cost. However, it mainly optimizes operations between matrices and matrices rather than operations between matrices and data/parity block vectors. The cost of matrices and matrices operations is much smaller than that of matrices and block vectors operations in most scenarios. Thus, the optimization is limited and decreases as the block size increases. However, our PPM and gPPM algorithms mainly optimize the operation of matrices and data/parity block vectors, so this problem does not exist.

Exploiting the parallelism during the encoding/decoding process has been a longstanding goal of storage systems design. A lot of parallelism algorithms, such as block-level parallelism algorithms [18, 28, 35, 52], disk-level parallelism algorithms [17, 32, 51], node-level parallelism algorithms [30, 33, 36], and equation-oriented parallelism algorithms [50] have been presented. Different from these algorithms, PPM is a matrix-oriented parallelism algorithm and extracts independent sub-matrices based on the independent faulty blocks, which can reduce the computational cost, as shown in Section 2.3. The equation-oriented parallelism algorithm presented in Reference [50] is closer to PPM. Still, it is designed for the Cauchy RS code (a kind of symmetric parity erasure code) and only discusses it in theory. At the same time, gPPM is a general algorithm for asymmetric and symmetric parity erasure codes. Besides, PPM/gPPM can also reduce computational cost by selecting the optimal matrix calculation sequence except for exploiting parallelism.

7 CONCLUSION

In this article, we categorize erasure codes into two categories: symmetric parity erasure codes and asymmetric parity erasure codes, based on whether the same number of blocks calculates each parity block. Our analysis shows that some state-of-the-art asymmetric parity erasure codes still use the traditional parity-check matrix to conduct the encoding/decoding process. By analyzing the process, we find that the independent faulty blocks provide a chance to exploit parallelism and reduce computational cost. It motivates us to present an optimization algorithm for these codes.

The optimization algorithm, called the PPM algorithm, can improve the encoding/decoding speed by decreasing the computational cost and exploiting the parallelism of the encoding/decoding operations. When applied to SD code, PMDS code, and LRC code, PPM achieves up to a 210.81% improvement in the encoding/decoding speed for various configurations, thread numbers, stripe sizes, and CPUs. Furthermore, we demonstrate that symmetric parity erasure codes can also be optimized by selecting the calculation sequence with the minimum computational cost. This insight motivates us to derive a gPPM algorithm from the PPM algorithm, which improves over PPM by up to 102.41% and can be applied to symmetric parity erasure codes to improve the encoding/decoding performance.

ACKNOWLEDGMENT

Thanks for the anonymous reviewers' insightful comments and valuable suggestions.

REFERENCES

- [1] 2017. Ultra-Low Latency with Samsung ZNAND SSD. Retrieved from https://www.samsung.com/semiconductor/global.semi.static/Ultra-Low_Latency_with_Samsung_Z-NAND_SSD-0.pdf
- [2] 2023. Amazon EC2. Retrieved from <http://aws.amazon.com/ec2/>
- [3] 2023. Intel® Intelligent Storage Acceleration Library (Intel® ISA-L). Retrieved from <https://www.intel.com/content/www/us/en/developer/tools/isa-l/overview.html>
- [4] 2023. Intel® Optane™ SSD DC P5800X Series. Retrieved from <https://ark.intel.com/content/www/us/en/ark/products/201859/intel-optane-ssd-dc-p5800x-series-1-6tb-2-5in-pcie-x4-3d-xpoint.html>
- [5] 2023. Optimizing Software for the Next Gen Intel Optane SSD P5800X. Retrieved from <https://www.intel.com/content/www/us/en/events/memory-and-storage.html?videoId=6215534787001>
- [6] H. P. Anvin. 2011. The mathematics of RAID-6. In Retrieved from <http://kernel.org/pub/linux/kernel/people/hpa/raid6.pdf>
- [7] Lakshmi N. Bairavasundaram, Garth R. Goodson, Shankar Pasupathy, and Jiri Schindler. 2007. An analysis of latent sector errors in disk drives. In *Proceedings of the SIGMETRICS 2007*. San Diego, California.
- [8] Lakshmi N. Bairavasundaram, Garth R. Goodson, Bianca Schroeder, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2008. An analysis of data corruption in the storage stack. In *Proceedings of the 6th Usenix Conference on File and Storage Technologies*.
- [9] M. Blaum, J. Brady, J. Bruck, and J. Menon. 1995. EVENODD: An efficient scheme for tolerating double disk failures in RAID architectures. *IEEE Transactions on Computers* 44, 2 (February 1995).
- [10] M. Blaum, J. L. Hafner, and S. Hetzler. 2012. *Partail-MDS Codes and their Application to RAID Type of Architectures*. Technical Report RJ10498. IBM Research Report.
- [11] J. Blomer, M. Kalfane, R. Karp, M. Karpinski, M. Luby, and D. Zuckerman. 1995. *An XOR-Based Erasure-Resilient Coding Scheme*. Technical Report TR-95-048. International Computer Science Institute.
- [12] B. Calder, J. Wang, A. Ogus, and N. Nilakantan et al. 2011. Windows azure storage: A highly available cloud storage service with strong consistency. In *Proceedings of the 23rd ACM Symposium on Operating Systems Principles*.
- [13] P. Corbett, B. English, A. Goel, T. Grcanac, S. Kleiman, J. Leong, and S. Sankar. 2004. Row-diagonal parity for double disk failure correction. In *Proceedings of the 3rd Usenix Conference on File and Storage Technologies*. San Francisco, CA.
- [14] Daniel Ford, François Labelle, Florentina I. Popovici, Murray Stokely, Van-Anh Truong, Luiz Barroso, Carrie Grimes, and Sean Quinlan. 2010. Availability in globally distributed storage systems. In *Proceedings of 9th USENIX Symposium on Operating systems Design and Implementation*.
- [15] Junqing Gu, Chentao Wu, Xin Xie, Han Qiu, Jie Li, Minyi Guo, Xubin He, Yuanyuan Dong, and Yafei Zhao. 2019. Optimizing the parity check matrix for efficient decoding of rs-based cloud storage systems. In *Proceedings of the IPDPS'19*.
- [16] W. Haddock, P. V. Bangalore, M. L. Curry, and A. Skjellum. 2019. High performance erasure coding for very large stripe sizes. In *Proceedings of IEEE SpringSim*.
- [17] Mark Holland, Garth A. Gibson, and Daniel P. Siewiorek. 1993. Fast, on-line failure recovery in redundant disk arrays. In *Proceedings of the 23rd International Symposium on Fault-Tolerant Computing*.
- [18] R. Hou, J. Menon, and Y. Patt. 1993. Balancing I/O response time and disk rebuild time in a RAID5 disk array. In *Proceedings of the 26th Hawaii International Conference on System Sciences*. Kihei, HI.
- [19] Yuchong Hu, Henry C. H. Chen, Patrick P.C. Lee, and Yang Tang. 2012. NCCloud: Applying network coding for the storage repair in a cloud-of-clouds. In *Proceedings of the 10th USENIX Conference on File and Storage Technologies*. San Jose, CA.

- [20] Y. Hu, L. Cheng, Q. Yao, P. P. Lee, W. Wang, and W. Chen. 2021. Exploiting combined locality for Wide-Stripe erasure coding in distributed storage. In *Proceedings of the 19th Usenix Conference on File and Storage Technologies*.
- [21] Cheng Huang, Huseyin Simitci, Yikang Xu, Aaron Ogus, Brad Calder, Parikshit Gopalan, Jin Li, and Sergey Yekhanin. 2012. Erasure coding in windows azure storage. In *Proceedings of the 2012 USENIX Annual Technical Conference*.
- [22] C. Huang and L. Xu. 2005. STAR: An efficient coding scheme for correcting triple storage node failures. In *Proceedings of the 4th USENIX Conference on File and Storage Technologies*. San Francisco, CA.
- [23] Ilias Iliadis, Robert Haas, Xiao-Yu Hu, and Evangelos Eleftheriou. 2008. Disk scrubbing versus intra-disk redundancy for high-reliability RAID storage systems. In *Proceedings of the SIGMETRICS 2008*.
- [24] Weihang Jiang, Chongfeng Hu, Yuanyuan Zhou, and Arkady Kanevsky. 2008. Are disks the dominant contributor for storage failures? A comprehensive study of storage subsystem failure characteristics. In *Proceedings of the 6th Usenix Conference on File and Storage Technologies*.
- [25] Osama Khan, Randal Burns, James Plank, William Pierce, and Cheng Huang. 2012. Rethinking erasure codes for cloud file systems: Minimizing I/O for recovery and degraded reads. In *Proceedings of the 10th USENIX Conference on File and Storage Technologies*. San Jose, CA.
- [26] Hyojun Kim, Sangeetha Seshadri, Clement L. Dickey, and Lawrence Chiu. 2014. Evaluating phase change memory for enterprise storage systems: A study of caching and tiering approaches. In *Proceedings of the 12th USENIX Conference on File and Storage Technologies*. Santa Clara, CA.
- [27] B. C. Lee, E. Ipek, O. Mutlu, and D. Burger. 2009. Architecting phase change memory as a scalable DRAM alternative. In *Proceedings of the 36th International Symposium on Computer Architecture*.
- [28] J. Lee and J. Lui. 2002. Automatic recovery from disk failure in continuous-media servers. *IEEE Transactions on Parallel and Distributed Systems* 13, 5 (May 2002), 499–515.
- [29] Mingqiang Li and Patrick P. C. Lee. 2014. STAIR codes: A general family of erasure codes for tolerating device and sector failures in practical storage systems. In *Proceedings of the 12th USENIX Conference on File and Storage Technologies*. Santa Clara, CA, 147–162.
- [30] R. Li, X. Li, P. P. C. Lee, and Q. Huang. 2017. Repair pipelining for erasure-coded storage. In *Proceedings of USENIX ATC*.
- [31] Shiyi Li, Qiang Cao, Shenggang Wan, Wenhui Zhang, Changsheng Xie, Xubin He, and Pradeep Subedi. 2015. PPM: A partitioned and parallel matrix algorithm to accelerate encoding/decoding process of asymmetric parity erasure codes. In *Proceedings of the ICPP 2015*.
- [32] Y. Li, N. Wang, C. Tian, S. Wu, Y. Zhang, and Y. Xu. 2018. A hierarchical RAID architecture towards fast recovery and high reliability. *IEEE Transactions on Parallel and Distributed Systems* 29, 4 (April 2018), 734–747.
- [33] S. Lin, G. Gong, Z. Shen, P. P. Lee, and J. Shu. 2021. Boosting full-node repair in erasure-coded storage. In *Proceedings of USENIX ATC*.
- [34] F. J. MacWilliams and N. J. A. Sloane. 1977. The theory of error-correcting codes, part I. In *Proceedings of the North-Holland Publishing Company*. New York.
- [35] J. Menon and D. Mattson. 1992. Distributed sparing in disk arrays. In *Proceedings of the 37th International Conference on COMPCON*. San Francisco, California, 410–421.
- [36] S. Mitra, R. Panta, M. Ra, and S. Bagchi. 2016. Partial-parallel-repair (PPR): A distributed technique for repairing erasure coded storage. In *Proceedings of ACM EuroSys*.
- [37] W. W. Peterson and E. J. Weldon Jr. 1972. *Error-Correcting Codes* (2nd ed). MIT Press. Cambridge.
- [38] Eduardo Pinheiro, Wolf-Dietrich Weber, and Luiz Andre Barroso. 2007. Failure trends in a large disk drive population. In *Proceedings of the 5th Usenix Conference on File and Storage Technologies*.
- [39] J. Plank. 2008. The RAID-6 liberation codes. In *Proceedings of the 6th USENIX Conference on File and Storage Technologies*.
- [40] J. S. Plank. 2013. Open source encoder and decoder for SD erasure codes. *Technical Report UT-CS-13- 704*.
- [41] J. S. Plank, M. Blaum, and J. L. Hafner. 2013. SD codes: Erasure codes designed for how storage systems really fail. In *Proceedings of the 11th Usenix Conference on File and Storage Technologies*.
- [42] J. S. Plank, K. M. Greenan, and E. L. Miller. 2013. Screaming fast galois field arithmetic using intel SIMD instructions. In *Proceedings of the 11th Usenix Conference on File and Storage Technologies*.
- [43] J. S. Plank, J. Luo, C. D. Schuman, L. Xu, and Z. Wilcox-O’Hearn. 2009. A performance evaluation and examination of open-source erasure coding libraries for storage. In *Proceedings of the 7th Usenix Conference on File and Storage Technologies*.
- [44] M. K. Qureshi, V. Srinivasan, and J. A. Rivers. 2009. Scalable high performance main memory system using phase-change memory technology. In *Proceedings of the 36th International Symposium on Computer Architecture*. 24–33.
- [45] RANDOM.ORG. 2022. Random integer generator. Retrieved from <http://www.random.org/integers/>
- [46] I. Reed and G. Solomon. 1960. Polynomial codes over certain finite fields. *Journal of the Society for Industrial and Applied Mathematics* 8, 2 (1960), 300–304.

- [47] M. Sathiamoorthy, Megasthenis Asteris, Dimitris Papailiopoulos, A. Dimakis, Ramkumar Vadali, Scott Chen, and Dhruba Borthakur. 2013. XORing elephants: Novel erasure codes for big data. In *Proceedings of the VLDB Endowment*, Vol. 6.
- [48] B. Schroeder, S. Damouras, and P. Gill. 2010. Understanding latent sector errors and how to protect against them. In *Proceedings of the 8th Usenix Conference on File and Storage Technologies*.
- [49] Bianca Schroeder and Garth A. Gibson. 2007. Disk failures in the real world: What does an MTTF of 1,000,000 hours mean to you?. In *Proceedings of the 5th Usenix Conference on File and Storage Technologies*.
- [50] Peter Sobe. 2010. Parallel reed/solomon coding on multicore processors. In *Proceedings of 2010 International Workshop on Storage Network Architecture and Parallel I/Os*.
- [51] L. Tian, D. Feng, H. Jiang, L. Zeng, J. Chen, Z. Wang, and Z. Song. 2007. PRO: A popularity-based multi-threaded reconstruction optimization for RAID-structured storage systems. In *Proceedings of the 5th USENIX Conference on File and Storage Technologies*.
- [52] Jiguang Wan, Jibin Wang, Changsheng Xie, and Qing Yang. 2014. S2-RAID: Parallel RAID architecture for fast data recovery. *IEEE Transactions on Parallel and Distributed Systems* 25, 6 (2014), 1638–1647.
- [53] Shenggang Wan, Xubin He, Jianzhong Huang, Qiang Cao, Shiyi Li, and Changsheng Xie. 2013. An efficient penalty aware cache to improve the performance of parity-based disk arrays under faulty conditions. *IEEE Transactions on Parallel and Distributed Systems* 24, 8 (2013), 1500–1513.
- [54] Si Wu, Qingpeng Du, Patrick P. C. Lee, Yongkun Li, and Yinlong Xu. 2021. StripeMerge: Efficient wide-stripe generation for large-scale erasure-coded storage. In *Proceedings of IEEE, ICDCS 2021*.
- [55] Q. Yao, Y. Hu, L. Cheng, P. P. Lee, D. Feng, W. Wang, and W. Chen. 2022. Optimal data placement for stripe merging in locally repairable codes. In *Proceedings of IEEE, INFOCOM 2022*.
- [56] Yuhong Zhong, Haoyu Li, Yu Jian Wu, Ioannis Zarkadas, Jeffrey Tao, Evan Mesterhazy, Michael Makris, Junfeng Yang, Amy Tai, Ryan Stutsman, and Asaf Cidon. 2022. XRP: In-kernel storage functions with eBPF. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI'22)*.

Received 17 May 2023; revised 31 July 2023; accepted 6 September 2023